In [1]:
```
import pandas as pd
```

In [2]:
```
from sklearn.datasets import fetch_openml
```

In [8]:
```
X, y = fetch_openml(data_id=1597, return_X_y=True)
```

In [9]:
```
len(X)
```

Out[9]:
```
284807
```

In [11]:
```
import numpy as np

# how many rows you want to keep
TARGET = 50000

# choose random row indices
sample_idx = np.random.choice(len(X), size=TARGET, replace=False)

# subset X and y
X = X.iloc[sample_idx].reset_index(drop=True)
y = y.iloc[sample_idx].reset_index(drop=True)
```

In [12]:
```
len(X)
```

Out[12]:
```
50000
```

In [13]:
```
X.shape
```

Out[13]:
```
(50000, 29)
```

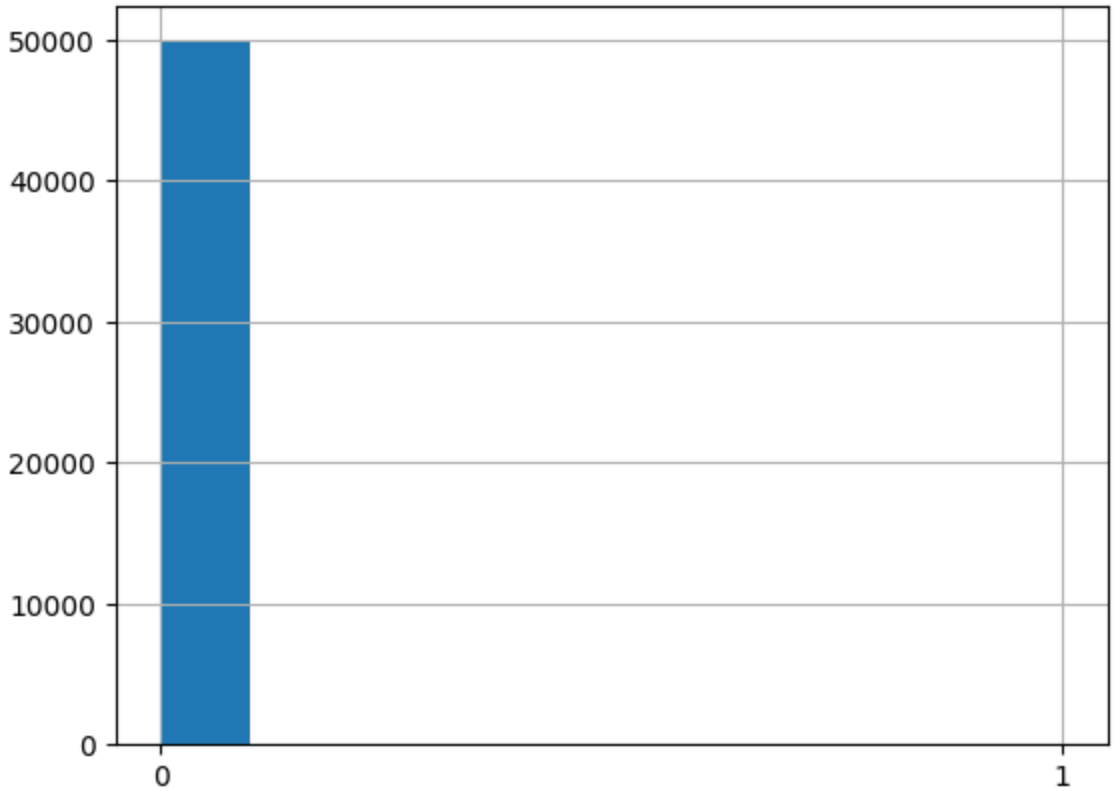
In [14]:
```
import numpy as np

np.unique(y)
```

Out[14]:
```
array(['0', '1'], dtype=object)
```

In [15]:
```
import seaborn as sns
```

In [16]:
```
pd.Series(y).hist()
```

Out[16]:
```
<Axes: >
```



In [17]:
```
X.mean(axis=0)
```

Out[17]:
```
V1        -0.006999
V2        -0.003187
V3         0.002979
V4        -0.009099
V5         0.010277
V6        -0.003877
V7        -0.000929
V8         0.000567
V9         0.000846
V10        0.007963
V11        0.004448
V12       -0.000191
V13       -0.002879
V14        0.000206
V15       -0.000786
V16       -0.000497
V17       -0.003936
V18       -0.002848
V19       -0.003398
V20        0.005579
V21        0.002683
V22       -0.002811
V23        0.000025
V24        0.001366
V25       -0.000792
V26        0.000662
V27        0.001573
V28        0.000192
Amount    87.447845
dtype: float64
```

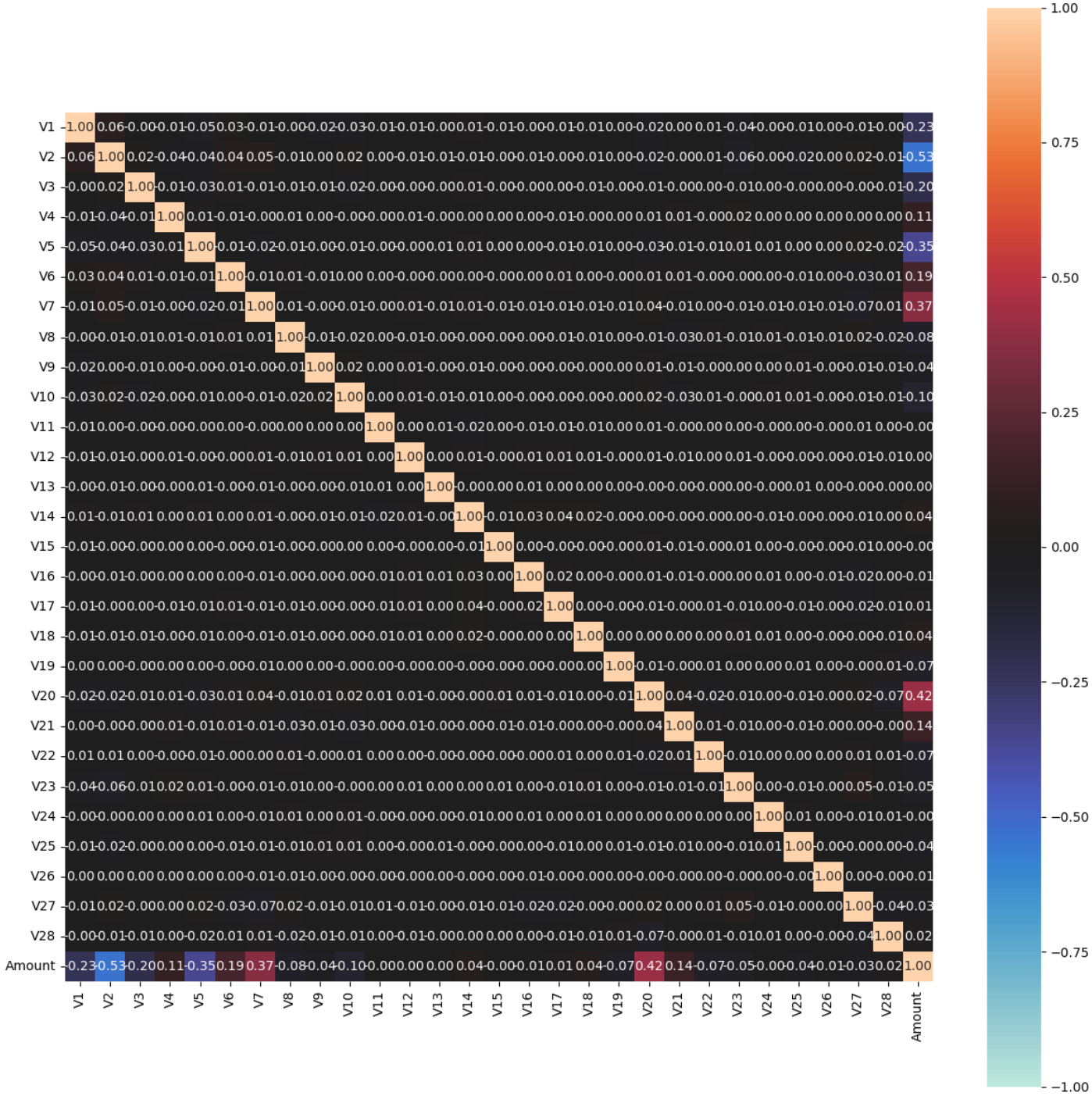
In [18]:
```
X.std(axis=0)
```

Out[18]:
```
V1          1.986700
V2          1.728987
V3          1.502779
V4          1.429239
V5          1.380196
V6          1.336650
V7          1.202854
V8          1.178617
V9          1.108098
V10         1.109827
V11         1.023947
V12         1.004622
V13         0.995224
V14         0.966327
V15         0.918632
V16         0.880104
V17         0.855060
V18         0.839712
V19         0.811469
V20         0.791462
V21         0.725558
V22         0.724935
V23         0.605523
V24         0.604846
V25         0.519720
V26         0.482291
V27         0.394383
V28         0.349242
Amount    244.984361
dtype: float64
```

In [19]:
```
from dython.nominal import associations
```

In [20]:
```
print('feature correlations')
associations(X, nominal_columns=None, figsize=[15, 15])
```

```
feature correlations
```


V1 1.00 0.06 -0.00 -0.01 -0.05 0.03 -0.01 -0.00 -0.02 -0.03 -0.01 -0.01 -0.00 0.01 -0.01 -0.00 -0.01 -0.01 0.00 -0.02 0.00 0.01 -0.04 -0.00 -0.01 0.00 -0.01 -0.00 -0.23
V2 0.06 1.00 0.02 -0.04 -0.04 0.04 0.05 -0.01 0.00 0.02 0.00 -0.01 -0.01 -0.01 -0.00 -0.01 -0.00 -0.01 0.00 -0.02 -0.00 0.01 -0.06 -0.00 -0.02 0.00 0.02 -0.01 -0.53
V3 -0.00 0.02 1.00 -0.01 -0.03 0.01 -0.01 -0.01 -0.01 -0.02 -0.00 -0.00 -0.00 0.01 -0.00 -0.00 0.00 -0.01 -0.00 -0.01 -0.00 0.00 -0.01 0.00 -0.00 0.00 -0.00 -0.01 -0.20
V4 -0.01 -0.04 -0.01 1.00 0.01 -0.01 -0.00 0.01 0.00 -0.00 -0.00 0.01 -0.00 0.00 0.00 0.00 -0.01 -0.00 0.00 0.01 0.01 -0.00 0.02 0.00 0.00 0.00 0.00 0.00 0.11
V5 -0.05 -0.04 -0.03 0.01 1.00 -0.01 -0.02 -0.01 -0.00 -0.01 -0.00 -0.00 0.01 0.01 0.00 0.00 -0.01 -0.01 0.00 -0.03 -0.01 -0.01 0.01 0.01 0.00 0.00 0.02 -0.02 -0.35
V6 0.03 0.04 0.01 -0.01 -0.01 1.00 -0.01 0.01 -0.01 0.00 0.00 -0.00 -0.00 0.00 -0.00 0.00 0.01 0.00 -0.00 0.01 0.01 -0.00 -0.00 0.00 -0.01 0.00 -0.03 0.01 0.19
V7 -0.01 0.05 -0.01 -0.00 -0.02 -0.01 1.00 0.01 -0.00 -0.01 -0.00 0.01 -0.01 0.01 -0.01 -0.01 -0.01 -0.01 -0.01 0.04 -0.01 0.00 -0.01 -0.01 -0.01 -0.01 -0.07 0.01 0.37
V8 -0.00 -0.01 -0.01 0.01 -0.01 0.01 0.01 1.00 -0.01 -0.02 0.00 -0.01 -0.00 0.00 -0.00 -0.00 -0.01 -0.01 0.00 -0.01 -0.03 0.01 -0.01 0.01 -0.01 -0.01 0.02 -0.02 -0.08
V9 -0.02 0.00 -0.01 0.00 -0.00 -0.01 -0.00 -0.01 1.00 0.02 0.00 0.01 -0.00 -0.01 -0.00 -0.00 -0.00 -0.00 0.00 0.01 -0.01 -0.00 0.00 0.00 0.01 -0.00 -0.01 -0.01 -0.04
V10 -0.03 0.02 -0.02 -0.00 -0.01 0.00 -0.01 -0.02 0.02 1.00 0.00 0.01 -0.01 -0.01 0.00 -0.00 -0.00 -0.00 -0.00 0.02 -0.03 0.01 -0.00 0.01 0.01 -0.00 -0.01 -0.01 -0.10
V11 -0.01 0.00 -0.00 -0.00 -0.00 0.00 -0.00 0.00 0.00 0.00 1.00 0.00 0.01 -0.02 0.00 -0.01 -0.01 -0.01 0.00 0.01 -0.00 0.00 0.00 -0.00 0.00 -0.00 0.01 0.00 -0.00
V12 -0.01 -0.01 -0.00 0.01 -0.00 -0.00 0.01 -0.01 0.01 0.01 0.00 1.00 0.00 0.01 -0.00 0.01 0.01 0.01 -0.00 0.01 -0.01 0.00 0.01 -0.00 -0.00 -0.00 -0.01 -0.01 0.00
V13 -0.00 -0.01 -0.00 -0.00 0.01 -0.00 -0.01 -0.00 -0.00 -0.01 0.01 0.00 1.00 -0.00 0.00 0.01 0.00 0.00 0.00 -0.00 -0.00 -0.00 0.00 -0.00 0.01 0.00 -0.00 -0.00 0.00
V14 -0.01 -0.01 0.01 0.00 0.01 0.00 0.01 -0.00 -0.01 -0.01 -0.02 0.01 -0.00 1.00 -0.01 0.03 0.04 0.02 -0.00 -0.00 -0.00 -0.00 0.00 -0.01 -0.00 -0.00 -0.01 0.00 0.04
V15 -0.01 -0.00 -0.00 0.00 0.00 -0.00 -0.01 -0.00 -0.00 0.00 0.00 -0.00 0.00 -0.01 1.00 0.00 -0.00 -0.00 -0.00 0.01 -0.01 -0.00 0.01 0.00 -0.00 -0.00 -0.01 0.00 -0.00
V16 -0.00 -0.01 -0.00 0.00 0.00 0.00 -0.01 -0.00 -0.00 -0.00 -0.01 0.01 0.01 0.03 0.00 1.00 0.02 0.00 -0.00 0.01 -0.01 -0.00 0.00 0.01 0.00 -0.01 -0.02 0.00 -0.01
V17 -0.01 -0.00 0.00 -0.01 -0.01 0.01 -0.01 -0.01 -0.00 -0.00 -0.01 0.01 0.00 0.04 -0.00 0.02 1.00 0.00 -0.00 -0.01 -0.00 0.01 -0.01 0.00 -0.01 -0.00 -0.02 -0.01 0.01
V18 -0.01 -0.01 -0.01 -0.00 -0.01 0.00 -0.01 -0.01 -0.00 -0.00 -0.01 0.01 0.00 0.02 -0.00 0.00 0.00 1.00 0.00 0.00 0.00 0.00 0.01 0.01 0.00 -0.00 -0.00 -0.01 0.04
V19 0.00 0.00 -0.00 0.00 0.00 -0.00 -0.01 0.00 0.00 -0.00 0.00 -0.00 0.00 -0.00 -0.00 -0.00 -0.00 0.00 1.00 -0.01 -0.00 0.01 0.00 0.00 0.01 0.00 -0.00 0.01 -0.07
V20 -0.02 -0.02 -0.01 0.01 -0.03 0.01 0.04 -0.01 0.01 0.02 0.01 0.01 -0.00 -0.00 0.01 0.01 -0.01 0.00 -0.01 1.00 0.04 -0.02 -0.01 0.00 -0.01 -0.00 0.02 -0.07 0.42
V21 0.00 -0.00 -0.00 0.01 -0.01 0.01 -0.01 -0.03 -0.01 -0.03 0.00 -0.01 -0.00 -0.00 -0.01 -0.01 -0.00 0.00 -0.00 0.04 1.00 0.01 -0.01 0.00 -0.01 -0.00 0.00 -0.00 0.14
V22 0.01 0.01 0.00 -0.00 -0.01 -0.00 0.00 0.01 -0.00 0.01 0.00 0.00 -0.00 -0.00 -0.00 -0.00 0.01 0.00 0.01 -0.02 0.01 1.00 -0.01 0.00 0.00 0.00 0.01 0.01 -0.07
V23 -0.04 -0.06 -0.01 0.02 0.01 -0.00 -0.01 -0.01 0.00 -0.00 0.00 0.01 0.00 0.00 0.01 0.00 -0.01 0.01 0.00 -0.01 -0.01 -0.01 1.00 0.00 -0.01 -0.00 0.05 -0.01 -0.05
V24 -0.00 -0.00 0.00 0.00 0.01 0.00 -0.01 0.01 0.00 0.01 -0.00 -0.00 -0.00 -0.01 0.00 0.01 0.00 0.01 0.00 0.00 0.00 0.00 0.00 1.00 0.01 0.00 -0.01 0.01 -0.00
V25 -0.01 -0.02 -0.00 0.00 0.00 -0.01 -0.01 -0.01 0.01 0.01 0.00 -0.00 0.01 -0.00 -0.00 0.00 -0.01 0.00 0.01 -0.01 -0.01 0.00 -0.01 0.01 1.00 -0.00 -0.00 0.00 -0.04
V26 0.00 0.00 0.00 0.00 0.00 0.00 -0.01 -0.01 -0.00 -0.00 -0.00 -0.00 0.00 -0.00 0.00 -0.01 -0.00 -0.00 0.00 -0.00 -0.00 0.00 -0.00 0.00 -0.00 1.00 0.00 -0.00 -0.01
V27 -0.01 0.02 -0.00 0.00 0.02 -0.03 -0.07 0.02 -0.01 -0.01 0.01 -0.01 -0.00 -0.01 -0.01 -0.02 -0.02 -0.00 -0.00 0.02 0.00 0.01 0.05 -0.01 -0.00 0.00 1.00 -0.04 0.03
V28 -0.00 -0.01 -0.01 0.00 -0.02 0.01 0.01 -0.02 -0.01 -0.01 0.00 -0.01 -0.00 0.00 0.00 0.00 -0.01 -0.01 0.01 -0.07 -0.00 0.01 -0.01 0.01 0.00 -0.00 -0.04 1.00 0.02
Amount -0.23 -0.53 -0.20 0.11 -0.35 0.19 0.37 -0.08 -0.04 -0.10 -0.00 0.00 0.00 0.04 -0.00 -0.01 0.01 0.04 -0.07 0.42 0.14 -0.07 -0.05 -0.00 -0.04 -0.01 -0.03 0.02 1.00
V1 V2 V3 V4 V5 V6 V7 V8 V9 V10 V11 V12 V13 V14 V15 V16 V17 V18 V19 V20 V21 V22 V23 V24 V25 V26 V27 V28 Amount
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

Out[20]: {'corr':

	V1	V2	V3	V4	V5	V6	V7
V1	1.000000	0.064926	-0.002549	-0.006933	-0.054323	0.031704	-0.005740
V2	0.064926	1.000000	0.019793	-0.035153	-0.043005	0.040085	0.050458
V3	-0.002549	0.019793	1.000000	-0.005314	-0.031337	0.010687	-0.013793
V4	-0.006933	-0.035153	-0.005314	1.000000	0.010168	-0.010784	-0.003627
V5	-0.054323	-0.043005	-0.031337	0.010168	1.000000	-0.008449	-0.021700
V6	0.031704	0.040085	0.010687	-0.010784	-0.008449	1.000000	-0.006265
V7	-0.005740	0.050458	-0.013793	-0.003627	-0.021700	-0.006265	1.000000
V8	-0.003596	-0.013151	-0.012524	0.005466	-0.006627	0.009555	0.007273
V9	-0.019750	0.004192	-0.014936	0.000263	-0.001688	-0.009335	-0.004698
V10	-0.027809	0.023957	-0.018265	-0.004547	-0.010567	0.000984	-0.011497
V11	-0.007980	0.001781	-0.000671	-0.002610	-0.002195	0.000579	-0.004336
V12	-0.009620	-0.008387	-0.003508	0.005065	-0.000034	-0.002471	0.008936
V13	-0.000682	-0.009096	-0.003014	-0.001511	0.007062	-0.001856	-0.008014
V14	0.010290	-0.012936	0.009854	0.002115	0.008978	0.003248	0.014131
V15	-0.013755	-0.000065	-0.000071	0.003484	0.001447	-0.000998	-0.008856
V16	-0.003278	-0.005005	-0.000793	0.003499	0.000160	0.001878	-0.010741
V17	-0.007101	-0.003164	0.000055	-0.006091	-0.013945	0.009371	-0.012047
V18	-0.005768	-0.006257	-0.006797	-0.002186	-0.007042	0.003595	-0.005565
V19	0.004691	0.002618	-0.003596	0.003034	0.004915	-0.003720	-0.010101
V20	-0.023873	-0.016687	-0.010977	0.005655	-0.026522	0.013471	0.037631
V21	0.001496	-0.002652	-0.002760	0.008196	-0.012656	0.010884	-0.012405
V22	0.005288	0.005302	0.003059	-0.003987	-0.007226	-0.004521	0.003614
V23	-0.038200	-0.062972	-0.012725	0.023792	0.012174	-0.003796	-0.014453
V24	-0.000784	-0.001594	0.002622	0.001262	0.005942	0.003147	-0.007620
V25	-0.013511	-0.016722	-0.001785	0.002772	0.002782	-0.008420	-0.007668
V26	0.000053	0.001662	0.002536	0.001375	0.004836	0.001127	-0.008789
V27	-0.008723	0.020442	-0.002998	0.003378	0.019865	-0.026446	-0.066061
V28	-0.003636	-0.008731	-0.005721	0.004796	-0.015066	0.006850	0.014746
Amount	-0.230470	-0.532559	-0.198675	0.112885	-0.354178	0.185268	0.371632

	V8	V9	V10	...	V20	V21	V22
V1	-0.003596	-0.019750	-0.027809	...	-0.023873	0.001496	0.005288
V2	-0.013151	0.004192	0.023957	...	-0.016687	-0.002652	0.005302
V3	-0.012524	-0.014936	-0.018265	...	-0.010977	-0.002760	0.003059
V4	0.005466	0.000263	-0.004547	...	0.005655	0.008196	-0.003987
V5	-0.006627	-0.001688	-0.010567	...	-0.026522	-0.012656	-0.007226
V6	0.009555	-0.009335	0.000984	...	0.013471	0.010884	-0.004521
V7	0.007273	-0.004698	-0.011497	...	0.037631	-0.012405	0.003614
V8	1.000000	-0.007966	-0.015064	...	-0.006143	-0.026934	0.011148
V9	-0.007966	1.000000	0.019728	...	0.009670	-0.012679	-0.002328
V10	-0.015064	0.019728	1.000000	...	0.017870	-0.032059	0.007453
V11	0.003740	0.004104	0.002639	...	0.006417	-0.000776	0.003328
V12	-0.008377	0.008182	0.012187	...	0.007668	-0.010269	0.001397
V13	-0.003639	-0.002409	-0.007773	...	-0.003184	-0.000477	-0.000786
V14	-0.004266	-0.005470	-0.008600	...	-0.003309	-0.001256	-0.004966
V15	-0.001140	-0.001678	0.002219	...	0.010632	-0.006042	-0.004408
V16	-0.002384	-0.002343	-0.003041	...	0.006328	-0.008626	-0.003313
V17	-0.011272	-0.000078	-0.001966	...	-0.005138	-0.004280	0.007180
V18	-0.005392	-0.000716	-0.004519	...	0.000354	0.003300	0.001112
V19	0.004050	0.003643	-0.004301	...	-0.008508	-0.003062	0.005624
V20	-0.006143	0.009670	0.017870	...	1.000000	0.035006	-0.017275
V21	-0.026934	-0.012679	-0.032059	...	0.035006	1.000000	0.010401
V22	0.011148	-0.002328	0.007453	...	-0.017275	0.010401	1.000000
V23	-0.013602	0.001242	-0.004780	...	-0.014453	-0.013762	-0.013763
V24	0.008838	0.000988	0.010245	...	0.001757	0.001729	0.002492
V25	-0.008330	0.007866	0.011316	...	-0.010017	-0.014091	0.000684
V26	-0.006362	-0.002936	-0.001757	...	-0.001448	-0.004408	0.000811
V27	0.018821	-0.008074	-0.013137	...	0.020941	0.002746	0.013015
V28	-0.016823	-0.007820	-0.014360	...	-0.068589	-0.004570	0.006704
Amount	-0.077418	-0.042448	-0.104072	...	0.417642	0.135971	-0.069336

```
              V23       V24       V25       V26       V27       V28    Amount
V1     -0.038200 -0.000784 -0.013511  0.000053 -0.008723 -0.003636 -0.230470
V2     -0.062972 -0.001594 -0.016722  0.001662  0.020442 -0.008731 -0.532559
V3     -0.012725  0.002622 -0.001785  0.002536 -0.002998 -0.005721 -0.198675
V4      0.023792  0.001262  0.002772  0.001375  0.003378  0.004796  0.112885
V5      0.012174  0.005942  0.002782  0.004836  0.019865 -0.015066 -0.354178
V6     -0.003796  0.003147 -0.008420  0.001127 -0.026446  0.006850  0.185268
V7     -0.014453 -0.007620 -0.007668 -0.008789 -0.066061  0.014746  0.371632
V8     -0.013602  0.008838 -0.008330 -0.006362  0.018821 -0.016823 -0.077418
V9      0.001242  0.000988  0.007866 -0.002936 -0.008074 -0.007820 -0.042448
V10    -0.004780  0.010245  0.011316 -0.001757 -0.013137 -0.014360 -0.104072
V11     0.001367 -0.001524  0.004930 -0.003655  0.006084  0.001477 -0.000656
V12     0.006257 -0.000161 -0.003256 -0.000096 -0.010569 -0.007669  0.002063
V13     0.000953 -0.001553  0.005958  0.004023 -0.000206 -0.001493  0.002752
V14     0.003510 -0.005027 -0.004606 -0.002442 -0.010292  0.001230  0.042855
V15     0.006695  0.004936 -0.002439 -0.003482 -0.006355  0.004472 -0.001766
V16     0.001989  0.006970  0.003358 -0.005042 -0.023995  0.003868 -0.007437
V17    -0.008475  0.003080 -0.007346 -0.002091 -0.017883 -0.009972  0.012418
V18     0.007870  0.007332  0.003041 -0.000437 -0.003809 -0.009192  0.044670
V19     0.000265  0.004543  0.006545  0.002942 -0.001381  0.013918 -0.065725
V20    -0.014453  0.001757 -0.010017 -0.001448  0.020941 -0.068589  0.417642
V21    -0.013762  0.001729 -0.014091 -0.004408  0.002746 -0.004570  0.135971
V22    -0.013763  0.002492  0.000684  0.000811  0.013015  0.006704 -0.069336
V23     1.000000  0.000551 -0.006170 -0.000100  0.049005 -0.006179 -0.054888
V24     0.000551  1.000000  0.011469  0.001752 -0.007660  0.005177 -0.003103
V25    -0.006170  0.011469  1.000000 -0.002212 -0.003038  0.004524 -0.042867
V26    -0.000100  0.001752 -0.002212  1.000000  0.000086 -0.003116 -0.010211
V27     0.049005 -0.007660 -0.003038  0.000086  1.000000 -0.043985 -0.027985
V28    -0.006179  0.005177  0.004524 -0.003116 -0.043985  1.000000  0.015354
Amount -0.054888 -0.003103 -0.042867 -0.010211 -0.027985  0.015354  1.000000

[29 rows x 29 columns],
'ax': <Axes: >}
```

In [21]:

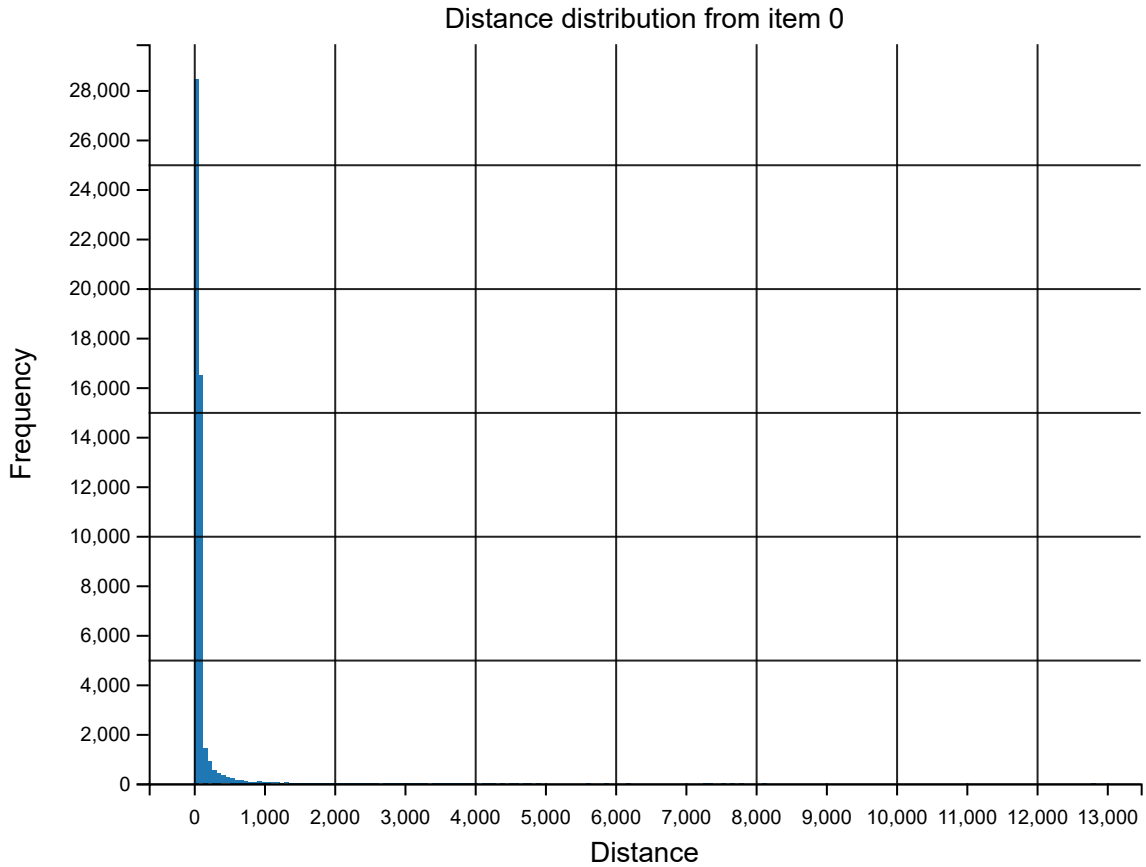
```
# from annoy import AnnoyIndex
# t = AnnoyIndex(X.shape[1], 'euclidean')  # Length of item vector that will be indexed
# for i, v in enumerate(X):
#     t.add_item(i, v)

# t.build(10) # 10 trees


# from sklearn.neighbors import NearestNeighbors
# import numpy as np

# nn = NearestNeighbors(
#     n_neighbors=10,
#     metric='euclidean',
#     algorithm='auto'
# )

# nn.fit(X)

# # Query example
# distances, indices = nn.kneighbors(X[0].reshape(1, -1), n_neighbors=10)
# print(indices, distances)

import numpy as np

def knn_bruteforce(X, query, k=10):
    dists = np.linalg.norm(X - query, axis=1)
```

```
    idx = np.argsort(dists)[:k]
    return idx, dists[idx]

# Convert DataFrame → NumPy
X_np = X.to_numpy().astype(float)

# Example
idx, dist = knn_bruteforce(X_np, X_np[0], k=10)
print(idx, dist)
```

```
[    0 31398 49179 29192 33387 10809 27946 17487 40902 11381] [0.         3.9557887  4.64401164 4
.69271115 4.94224672 5.03828428
 5.18402825 5.21134703 5.25145225 5.29096979]
```

In [22]:

```
# %matplotlib inline
# import mpld3
# mpld3.enable_notebook()

# _, distances = t.get_nns_by_item(0, 10000, include_distances=True)
# pd.Series(distances).hist(bins=200)


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import mpld3
mpld3.enable_notebook()

# Compute distances from item 0 to all items
query = X_np[0]
dists = np.linalg.norm(X_np - query, axis=1)

# Sort distances (Annoy returns sorted)
dists_sorted = np.sort(dists)

# Plot histogram
pd.Series(dists_sorted).hist(bins=200)
plt.title("Distance distribution from item 0")
plt.xlabel("Distance")
plt.ylabel("Frequency")
plt.show()
```



In [23]:
```
X.shape
```

Out[23]: (50000, 29)

In [24]:
```
import numpy as np
from sklearn.neighbors import NearestNeighbors
from joblib import Parallel, delayed
from tqdm import tqdm

import os
os.environ["LOKY_MAX_CPU_COUNT"] = "20"


# Convert DataFrame → NumPy
X_np = X.to_numpy().astype('float32')

# Parameters
MAX_NEIGHBORS = 300
THRESHOLD = 6.0
N_THREADS = 20
CHUNK_SIZE = 5000

# Build KNN index
nn = NearestNeighbors(
    n_neighbors=MAX_NEIGHBORS,
    metric='euclidean',
    algorithm='auto'
)
nn.fit(X_np)

# Function to compute neighbors for a block of rows
def process_chunk(start, end):
```

```
    chunk_result = {}
    for i in range(start, end):
        distances, indices = nn.kneighbors(
            X_np[i].reshape(1, -1),
            n_neighbors=MAX_NEIGHBORS
        )
        distances = distances[0]
        indices = indices[0]
        chunk_result[i] = [n for n, d in zip(indices, distances) if d < THRESHOLD]
    return chunk_result

# Create chunk boundaries
n_rows = X_np.shape[0]
chunks = [(i, min(i + CHUNK_SIZE, n_rows)) for i in range(0, n_rows, CHUNK_SIZE)]

# Run chunks in parallel with a global progress bar
results = []
for chunk_result in tqdm(
    Parallel(n_jobs=N_THREADS, backend="threading")(   # <-- FIXED HERE
        delayed(process_chunk)(start, end) for start, end in chunks
    ),
    total=len(chunks),
    desc="Processing chunks"
):
    results.append(chunk_result)

# Merge results
neighborhood = {}
for chunk in results:
    neighborhood.update(chunk)
##############################################
```

```
Processing chunks: 100%|
└└└
```

```
from tqdm import trange

n_rows = X_np.shape[0]
neighborhood = {}

for i in trange(n_rows):
    neighborhood[i] = get_neighbors(i)
```

```
100%|██████████████████████████████████████████████████████████| 50
000/50000 [02:28<00:00, 336.13it/s]
```

In [29]:
```
from scipy.sparse import lil_matrix
A = lil_matrix((n_rows, n_rows), dtype=np.int8)

for i, n in neighborhood.items():
  A[i, n] = 1.0
  A[n, i] = 1.0
```

In [30]:
```
print('max sparsity given max neighbors parameter: {}'.format(MAX_NEIGHBORS / n_rows))
```

```
max sparsity given max neighbors parameter: 0.006
```

In [31]:
```
print('average number of connections: {}'.format(A.sum(axis=0).mean()))
```

```
average number of connections: 316.64544
```

In [32]:
```
print('sparsity: {}'.format(A.sum() / (n_rows * n_rows)))
```

```
sparsity: 0.0063329088
```

In [34]:
```
import numpy as np
from sklearn.neighbors import NearestNeighbors
from joblib import Parallel, delayed
from tqdm import tqdm

# ---------------------------------------------------------
# 1. LOAD FULL DATASET
# ---------------------------------------------------------
from sklearn.datasets import fetch_openml
X, y = fetch_openml(data_id=1597, return_X_y=True)

# ---------------------------------------------------------
# 2. RANDOMLY REDUCE TO 50,000 ROWS
# ---------------------------------------------------------
TARGET = 50000
sample_idx = np.random.choice(len(X), size=TARGET, replace=False)

X = X.iloc[sample_idx].reset_index(drop=True)
y = y.iloc[sample_idx].reset_index(drop=True)

print("New dataset size:", len(X))

# ---------------------------------------------------------
# 3. CONVERT TO NUMPY
# ---------------------------------------------------------
X_np = X.to_numpy().astype('float32')

# ---------------------------------------------------------
# 4. PARAMETERS FOR NEIGHBOR GRAPH
# ---------------------------------------------------------
MAX_NEIGHBORS = 300
THRESHOLD = 6.0
```

```python
N_THREADS = 20
CHUNK_SIZE = 5000

# ---------------------------------------------------------
# 5. BUILD FAST KNN INDEX
# ---------------------------------------------------------
nn = NearestNeighbors(
    n_neighbors=MAX_NEIGHBORS,
    metric='euclidean',
    algorithm='auto'
)
nn.fit(X_np)

# ---------------------------------------------------------
# 6. FUNCTION TO PROCESS A CHUNK OF ROWS
# ---------------------------------------------------------
def process_chunk(start, end):
    chunk_result = {}
    for i in range(start, end):
        distances, indices = nn.kneighbors(
            X_np[i].reshape(1, -1),
            n_neighbors=MAX_NEIGHBORS
        )
        distances = distances[0]
        indices = indices[0]
        chunk_result[i] = [n for n, d in zip(indices, distances) if d < THRESHOLD]
    return chunk_result

# ---------------------------------------------------------
# 7. CREATE CHUNKS
# ---------------------------------------------------------
n_rows = X_np.shape[0]
chunks = [(i, min(i + CHUNK_SIZE, n_rows)) for i in range(0, n_rows, CHUNK_SIZE)]

# ---------------------------------------------------------
# 8. PARALLEL EXECUTION WITH VISIBLE PROGRESS
# ---------------------------------------------------------
results = []
for chunk_result in tqdm(
    Parallel(n_jobs=N_THREADS, backend="threading")(
        delayed(process_chunk)(start, end) for start, end in chunks
    ),
    total=len(chunks),
    desc="Processing chunks"
):
    results.append(chunk_result)

# ---------------------------------------------------------
# 9. MERGE RESULTS INTO FINAL NEIGHBORHOOD DICT
# ---------------------------------------------------------
neighborhood = {}
for chunk in results:
    neighborhood.update(chunk)

print("Finished building neighborhood graph.")
```

```
New dataset size: 50000
Processing chunks: 100%|
└└└
```

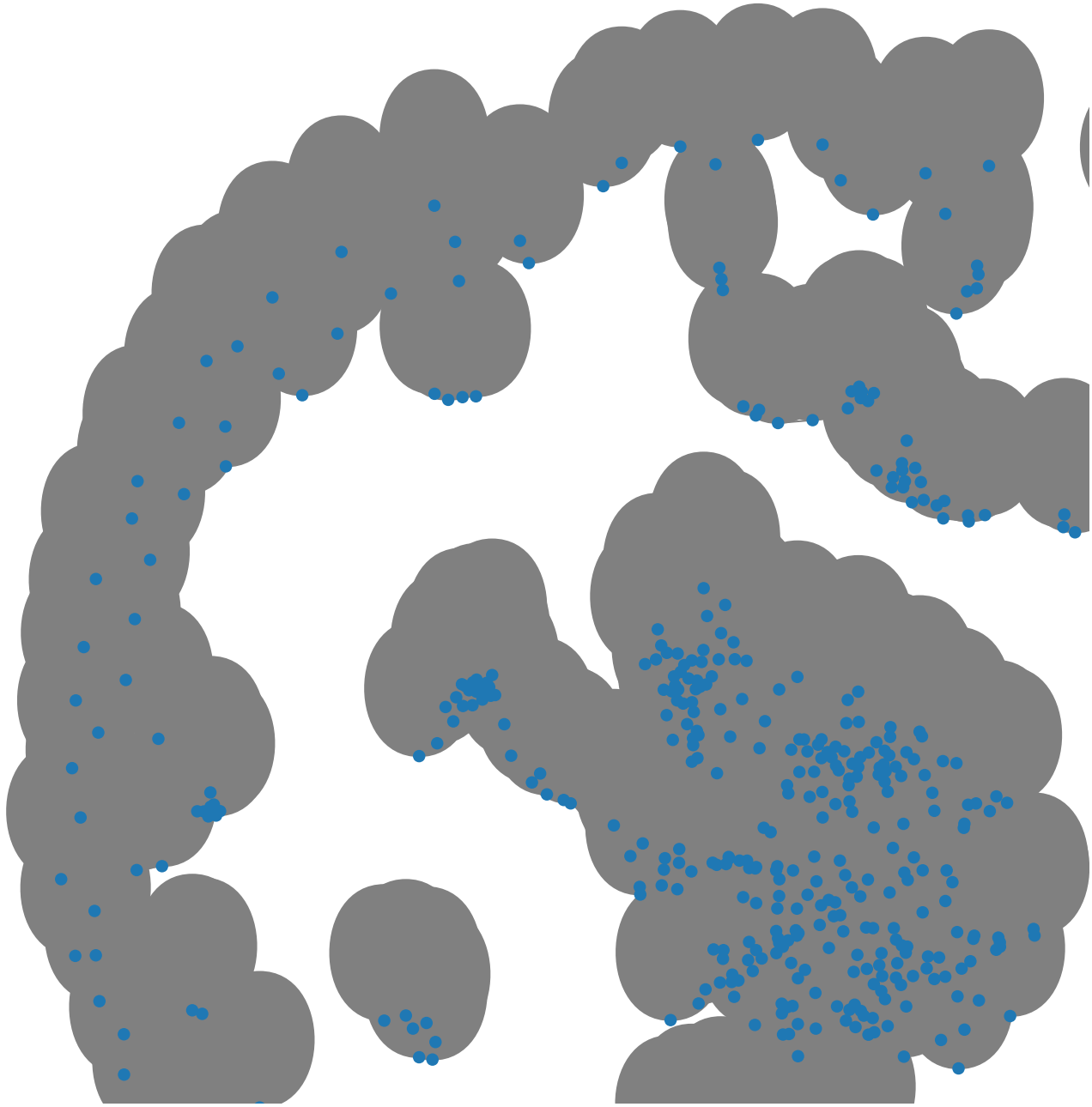
In [36]:
```
# import networkx as nx
# G = nx.from_scipy_sparse_matrix(A)

import networkx as nx
G = nx.from_scipy_sparse_array(A)
```

In [37]:
```
from scipy.sparse.csgraph import connected_components
n_components, labels = connected_components(A, directed=False, return_labels=True)
```

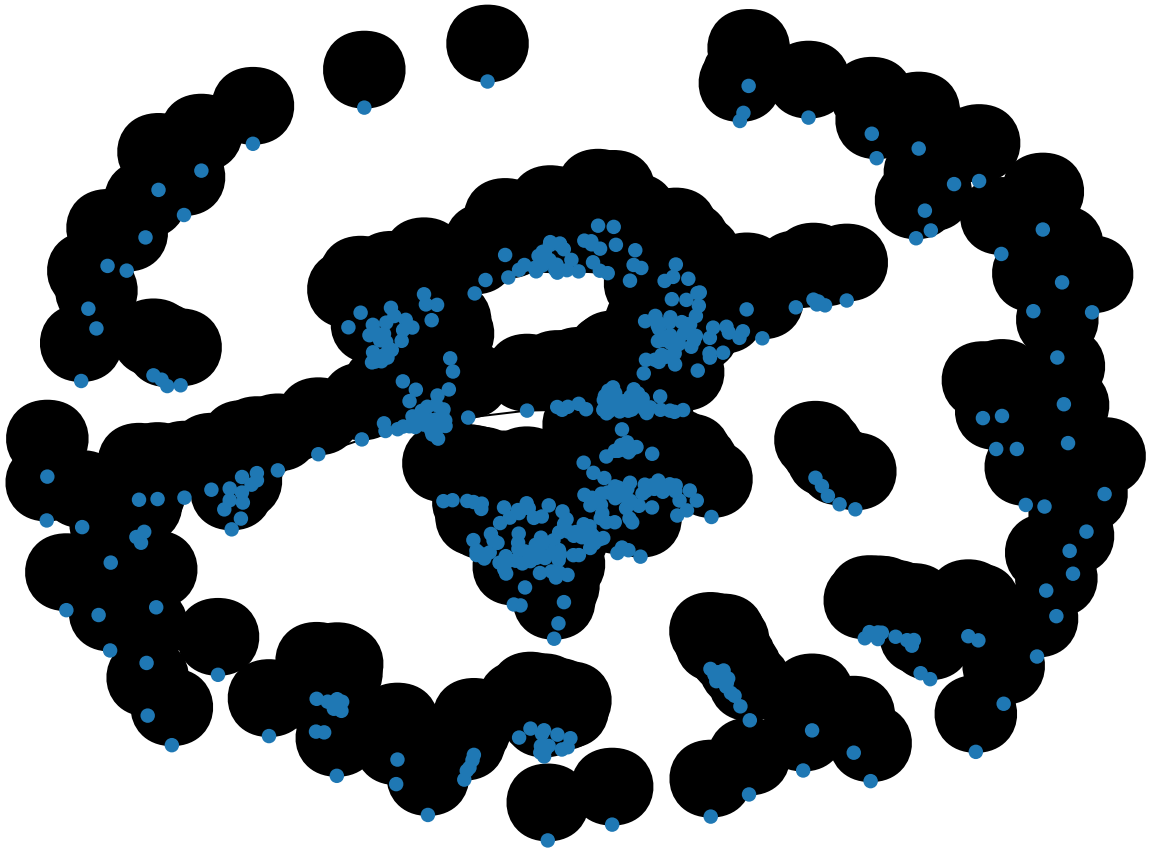
In [38]:
```
len(dists)
```

Out[38]:
```
50000
```

In [40]:
```
## nx.draw(G, with_labels=True, font_weight='bold')

#### reducing size for faster response

import networkx as nx
import matplotlib.pyplot as plt
import numpy as np

# sample 500 nodes
nodes = np.random.choice(G.nodes(), size=500, replace=False)
H = G.subgraph(nodes)

plt.figure(figsize=(10,10))
nx.draw(H, node_size=20, edge_color="gray")
plt.show()
```



In [41]:
```
largest_cc = max(nx.connected_components(G), key=len)
H = G.subgraph(largest_cc)

# sample 500 nodes from that component
nodes = np.random.choice(list(H.nodes()), size=500, replace=False)
H_small = H.subgraph(nodes)

nx.draw(H_small, node_size=20)
```



In [42]:
```
import numpy as np

# Euclidean distance (Annoy default)
def get_distance(i, j):
    return np.linalg.norm(X_np[i] - X_np[j])

# Squared Euclidean distance
def get_distance_sq(i, j):
    diff = X_np[i] - X_np[j]
    return np.dot(diff, diff)

# Cosine distance
def get_cosine_distance(i, j):
    u = X_np[i]
    v = X_np[j]
    return 1 - np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v))

# Angular distance (Annoy angular metric)
def get_angular_distance(i, j):
    u = X_np[i]
    v = X_np[j]
    cos_sim = np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v))
    return np.sqrt(2 * (1 - cos_sim))
```

In [43]:
```
get_distance(0, 25000)
```

Out[43]:
```
np.float32(90.79903)
```

In [44]:
```
get_distance(0, 30000)
```

Out[44]:
```
np.float32(15.877198)
```

In [47]:
```
##transaction_correlations = associations(X.transpose(), nominal_columns=None)


from sklearn.neighbors import NearestNeighbors

nn = NearestNeighbors(metric='euclidean', n_neighbors=50)
nn.fit(X_np)

distances, indices = nn.kneighbors(X_np)
```

In [51]:
```
#correlations.shape

#import pandas as pd
# Assuming 'df' is your DataFrame
correlations = X.corr()
print(correlations.shape)
```

```
(29, 29)
```

In [52]:
```
from scipy.spatial.distance import pdist
# https://docs.scipy.org/doc/scipy-0.14.0/reference/generated/scipy.spatial.distance.pdist.html
```

In [ ]:
```
# from sklearn.metrics import pairwise_distances

# correlations = pairwise_distances(X, metric='euclidean')
```

In [55]:
```
correlations = pdist(X, metric='euclidean')
```

In [56]:
```
correlations.shape
```

Out[56]:
```
(1249975000,)
```

In [59]:
```
pdist(X)
```

Out[59]:
```
array([ 24.6216177 ,  56.12209208,   4.34273872, ...,   6.83694117,
       350.24016397, 349.27534741])
```

In [57]:
```
import networkx as nx
```

In [61]:
```
## graph = nx.convert_matrix.from_pandas_edgelist(A, "id_1", "id_2")

import networkx as nx

# Convert LIL → CSR (NetworkX prefers CSR)
A_csr = A.tocsr()

# Build graph directly from sparse adjacency matrix
G = nx.from_scipy_sparse_array(A_csr)

print(G)
print(G.number_of_nodes(), "nodes")
print(G.number_of_edges(), "edges")
```

```
Graph with 50000 nodes and 7941136 edges
50000 nodes
7941136 edges
```

In [ ]: